1. Content analysis: Analyze the content of social media data to determine what topics are being given data set using topic modelling, keyword extractor

```
import pandas as pd
import nltk
import gensim
from gensim import corpora
from gensim.models import LdaModel
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from collections import Counter
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to C:\Users\Om
[nltk_data]     Shete\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\Om
[nltk_data]     Shete\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

```
df = pd.read_csv('CSV/social_media_data.csv')
print(df.head())
```

```
   id                                          text
0   1  Just had a great meal at the new restaurant do...
1   2  I love going to the beach with my family in th...
2   3  Can't wait to see the new movie coming out thi...
3   4  Feeling stressed out at work today, need a vac...
4   5         I can't believe how fast time is flying by
```

```
stop_words = set(stopwords.words('english'))
print("Stopwords: ")
print(stop_words)
lemmatizer = WordNetLemmatizer()
```

```
Stopwords:
{'aren', "it's", 'further', 'on', 'ourselves', 'under', 'himself', 'to', 'do', 'where', 'no', 'yourselves', 'needn', 'having', 'she
```

```
def pre_process(text):
    tokens = text.lower().split()
    lemma = [ lemmatizer.lemmatize(token) for token in tokens if token not in stop_words]
    return lemma
```

```
df['tokens'] = df['text'].apply(pre_process)
print(df.head())
```

```
   id  ...                                          tokens
0   1  ...          [great, meal, new, restaurant, downtown!]
1   2  ...              [love, going, beach, family, summer]
2   3  ...    [can't, wait, see, new, movie, coming, weekend]
3   4  ...  [feeling, stressed, work, today,, need, vacation]
4   5  ...               [can't, believe, fast, time, flying]

[5 rows x 3 columns]
```

```
# Topic Modeling with lda

dictionary = corpora.Dictionary(df['tokens'])
corpus = [dictionary.doc2bow(token) for token in df['tokens']]

num_topics = 5
lda_model = LdaModel(num_topics=num_topics, corpus=corpus, id2word=dictionary, passes=20)

print("Topics: ")
for topic_id in lda_model.print_topics():
    print(f"Topic: {topic_id}")
```

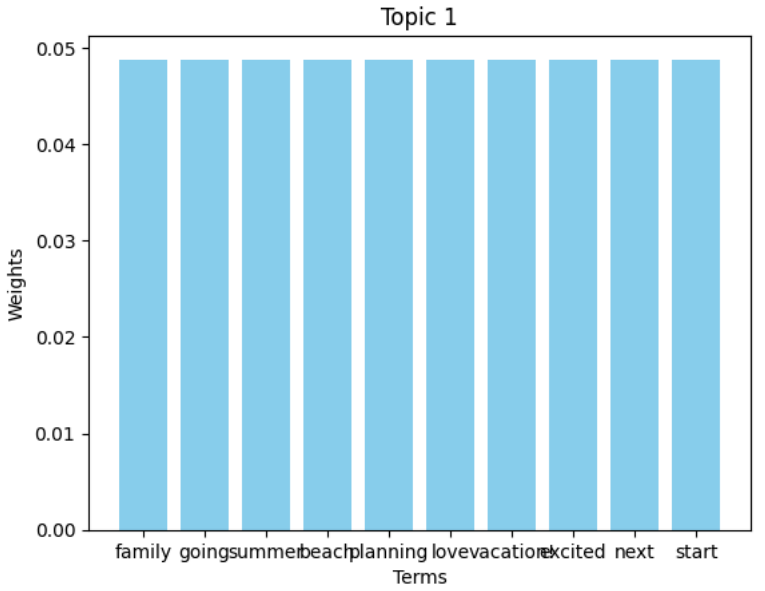
```
Topics:
Topic: (0, '0.049*"family" + 0.049*"going" + 0.049*"summer" + 0.049*"beach" + 0.049*"planning" + 0.049*"love" + 0.049*"vacation!" +
Topic: (1, '0.042*"believe" + 0.042*"live" + 0.042*"even" + 0.042*"amazing," + 0.042*"sounded" + 0.042*"band" + 0.042*"concert" + 0
Topic: (2, '0.062*"day" + 0.043*"feeling" + 0.043*"can\'t" + 0.043*"movie" + 0.043*"need" + 0.043*"today," + 0.023*"eating" + 0.023*
Topic: (3, '0.070*"great" + 0.038*"new" + 0.038*"highly" + 0.038*"finished" + 0.038*"reading" + 0.038*"it!" + 0.038*"book," + 0.038*
```

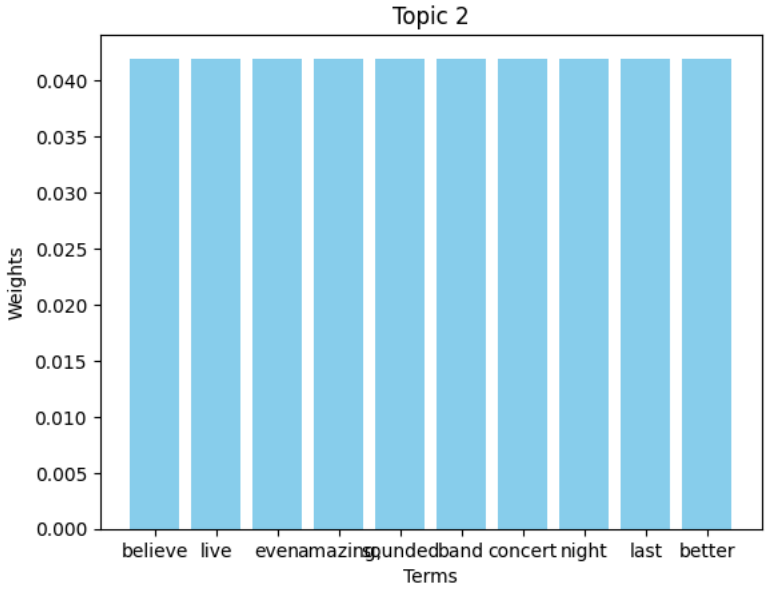
```
Topic: (4, '0.045*"new" + 0.045*"start" + 0.045*"next" + 0.045*"excited" + 0.045*"incredible" + 0.045*"mountains," + 0.045*"hiking"
```

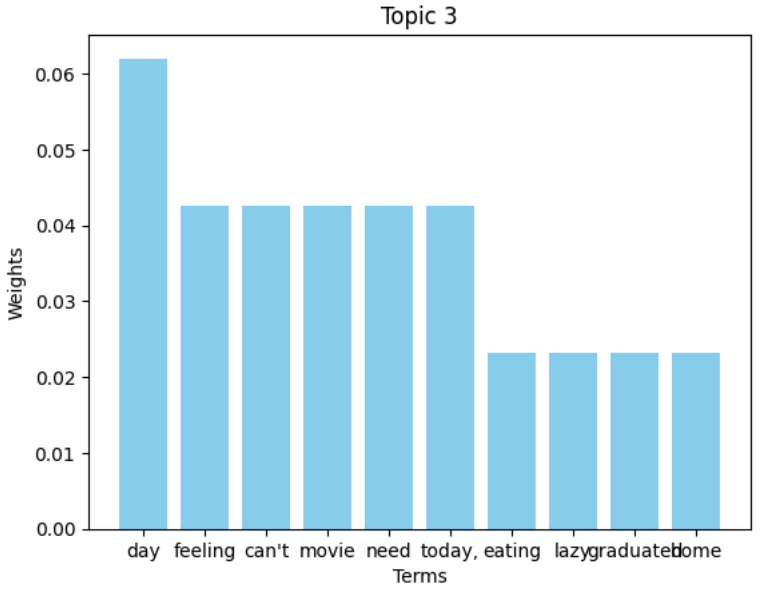
```
for idx in range(num_topics):
    topic_info = lda_model.print_topic(idx, topn=10)
    word_and_weights = topic_info.split(" + ")
    print(word_and_weights)

    for item in word_and_weights:
        word, weight = item.split("*")
        print(f" - {word.strip()} (Weight: {weight.strip()})")
```

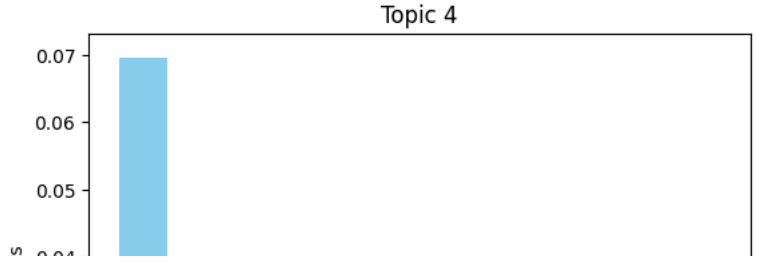
```
['0.049*"family"', '0.049*"going"', '0.049*"summer"', '0.049*"beach"', '0.049*"planning"', '0.049*"love"', '0.049*"vacation!"', '0.0
 - 0.049 (Weight: "family")
 - 0.049 (Weight: "going")
 - 0.049 (Weight: "summer")
 - 0.049 (Weight: "beach")
 - 0.049 (Weight: "planning")
 - 0.049 (Weight: "love")
 - 0.049 (Weight: "vacation!")
 - 0.049 (Weight: "excited")
 - 0.049 (Weight: "next")
 - 0.049 (Weight: "start")
['0.042*"believe"', '0.042*"live"', '0.042*"even"', '0.042*"amazing,"', '0.042*"sounded"', '0.042*"band"', '0.042*"concert"', '0.042
 - 0.042 (Weight: "believe")
 - 0.042 (Weight: "live")
 - 0.042 (Weight: "even")
 - 0.042 (Weight: "amazing,")
 - 0.042 (Weight: "sounded")
 - 0.042 (Weight: "band")
 - 0.042 (Weight: "concert")
 - 0.042 (Weight: "night")
 - 0.042 (Weight: "last")
 - 0.042 (Weight: "better")
['0.062*"day"', '0.043*"feeling"', '0.043*"can\'t"', '0.043*"movie"', '0.043*"need"', '0.043*"today,"', '0.023*"eating"', '0.023*"la
 - 0.062 (Weight: "day")
 - 0.043 (Weight: "feeling")
 - 0.043 (Weight: "can't")
 - 0.043 (Weight: "movie")
 - 0.043 (Weight: "need")
 - 0.043 (Weight: "today,")
 - 0.023 (Weight: "eating")
 - 0.023 (Weight: "lazy")
 - 0.023 (Weight: "graduated")
 - 0.023 (Weight: "home")
['0.070*"great"', '0.038*"new"', '0.038*"highly"', '0.038*"finished"', '0.038*"reading"', '0.038*"it!"', '0.038*"book,"', '0.038*"re
 - 0.070 (Weight: "great")
 - 0.038 (Weight: "new")
 - 0.038 (Weight: "highly")
 - 0.038 (Weight: "finished")
 - 0.038 (Weight: "reading")
 - 0.038 (Weight: "it!")
 - 0.038 (Weight: "book,")
 - 0.038 (Weight: "recommend")
 - 0.038 (Weight: "restaurant")
 - 0.038 (Weight: "downtown!")
['0.045*"new"', '0.045*"start"', '0.045*"next"', '0.045*"excited"', '0.045*"incredible"', '0.045*"mountains,"', '0.045*"hiking"', '0
 - 0.045 (Weight: "new")
 - 0.045 (Weight: "start")
 - 0.045 (Weight: "next")
 - 0.045 (Weight: "excited")
 - 0.045 (Weight: "incredible")
 - 0.045 (Weight: "mountains,")
 - 0.045 (Weight: "hiking")
 - 0.045 (Weight: "view")
 - 0.045 (Weight: "afternoon")
 - 0.045 (Weight: "spent")
```

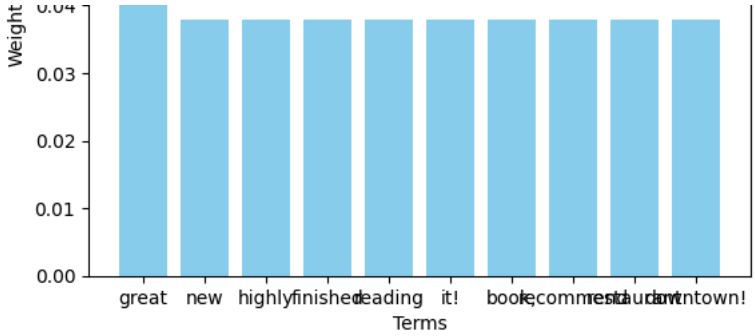
```
for idx, topic in enumerate(lda_model.show_topics(num_topics, formatted=False)):
    topic_term = [term[0] for term in topic[1]]
    term_weight = [term[1] for term in topic[1]]

    plt.figure()
    plt.bar(topic_term, term_weight, color='skyblue')
    plt.title(f"Topic {idx + 1}")
    plt.xlabel("Terms")
    plt.ylabel("Weights")
    plt.show()
```

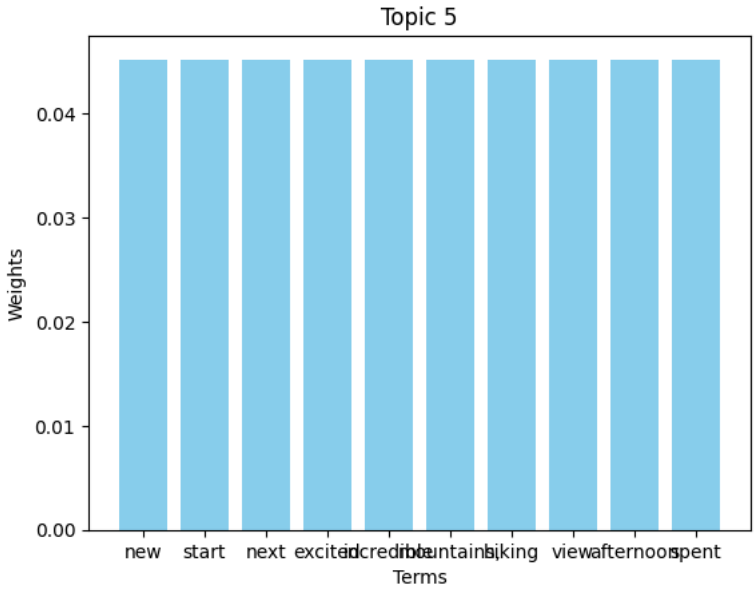

Topic 1
Weights
0.05
0.04
0.03
0.02
0.01
0.00
family
going
summer
beach
planning
love
vacation
excited
next
start
Terms


Topic 2
Weights
0.040
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
believe
live
even
amazing
sounded
band
concert
night
last
better
Terms


Topic 3
Weights
0.06
0.05
0.04
0.03
0.02
0.01
0.00
day
feeling
can't
movie
need
today,
eating
lazy
graduated
home
Terms


Topic 4
0.07
0.06
0.05






```
# keyword extraction
all_tokens = [ token for tokens in df['tokens'] for token in tokens]
keyword_ctr = Counter(all_tokens)
print("\nKeywords Counts: \n", keyword_ctr)

tfidf = TfidfVectorizer(max_features=20, stop_words='english')
x = tfidf.fit_transform(df['text'])

tfidf_keyword = tfidf.get_feature_names_out()
print("\nTop TF-IDF Keywords: ", tfidf_keyword)
```

```
Keywords Counts:
 Counter({'new': 3, "can't": 3, 'feeling': 3, 'day': 3, 'great': 2, 'family': 2, 'movie': 2, 'today,': 2, 'need': 2, 'believe': 2,

Top TF-IDF Keywords:  ['believe' 'day' 'excited' 'family' 'feeling' 'great' 'just' 'movies'
 'need' 'new' 'night' 'park' 'perfect' 'planning' 'popcorn' 'reading'
 'recommend' 'start' 'today' 'vacation']
```